

BOOK STORE

Full-Stack Web Application

Project Summary & Technical Specification

Tech Stack : React + Vite + Tailwind CSS | Node.js + Express + MongoDB

Auth : JWT (JSON Web Tokens)

Storage : MongoDB Atlas (Cloud) | Cloudinary (Images)

Date : May 2026

Prepared for the Book Store development project.

TABLE OF CONTENTS

1. Project Overview
2. Tech Stack
3. Database Models
4. API Endpoints
5. Frontend Pages & Features
6. Image Upload Flow (Cloudinary)
7. Folder Structure
8. Environment Variables
9. Key Deliverables

1. PROJECT OVERVIEW

A complete full-stack e-commerce book store where users can browse books, add them to favorites, manage a shopping cart, and place orders. Administrators have full control over book inventory, categories, and order management.

The application features a modern React frontend with Tailwind CSS styling, a robust Node.js/Express REST API backend, MongoDB Atlas for data persistence, and Cloudinary for image hosting. Authentication is handled via JWT tokens with role-based access control (user / admin).

Core Features

2. TECH STACK

Frontend

Technology	Purpose
React 18	UI library for building component-based interfaces
Vite	Fast development build tool with HMR
Tailwind CSS	Utility-first CSS framework for rapid styling
React Router v6	Client-side routing between pages
Axios	HTTP client for making API requests
React Context API	State management for auth, cart, and favorites

Backend

Technology	Purpose
Node.js	JavaScript runtime environment
Express	Web framework for building REST APIs
MongoDB Atlas	Cloud-hosted NoSQL database
Mongoose	ODM for MongoDB schema modeling
JWT (jsonwebtoken)	Authentication token generation & verification
bcryptjs	Password hashing and comparison
Multer	File upload parsing middleware
Cloudinary SDK	Cloud image upload, storage & delivery

3. DATABASE MODELS (MongoDB Collections)

User

Field	Type	Description
name	String (required)	User's display name
email	String (required, unique)	User's email address
password	String (required)	Hashed password (bcrypt)
role	String (enum)	user admin (default: user)
createdAt	Date	Auto-generated timestamp

Category

Field	Type	Description
name	String (required, unique)	Category name
description	String	Optional category description
createdAt	Date	Auto-generated timestamp

Book

Field	Type	Description
title	String (required)	Book title
author	String (required)	Book author
publisher	String (optional)	Publisher name
price	Number (required)	Price in currency units
quantity	Number (required)	Available stock count
category	ObjectId (ref)	Reference to Category collection
image	String	Cloudinary image URL
description	String	Book description
createdAt	Date	Auto-generated timestamp

Cart

Field	Type	Description
user	ObjectId (ref, unique)	Reference to User
items[].book	ObjectId (ref)	Reference to Book
items[].quantity	Number	Quantity of this book in cart

Order

Field	Type	Description
user	ObjectId (ref)	Reference to User who placed order

items[].book	ObjectId (ref)	Reference to Book
items[].quantity	Number	Quantity ordered
items[].price	Number	Price at time of order
totalAmount	Number	Total order amount
status	String (enum)	pending shipped delivered cancelled
createdAt	Date	Auto-generated timestamp

4. API ENDPOINTS

Authentication (/api/auth)

Method	Endpoint	Description	Access
POST	/api/auth/register	Create a new account	Public
POST	/api/auth/login	Login, returns JWT token	Public
GET	/api/auth/me	Get current user info	Auth

Books (/api/books)

Method	Endpoint	Description	Access
GET	/api/books	List all books (with filters & sort)	Public
GET	/api/books/:id	Get single book details	Public
POST	/api/books	Add new book (with image upload)	Admin
PUT	/api/books/:id	Update book details	Admin
DELETE	/api/books/:id	Delete a book	Admin

Categories (/api/categories)

Method	Endpoint	Description	Access
GET	/api/categories	List all categories	Public
POST	/api/categories	Add new category	Admin

Cart (/api/cart)

Method	Endpoint	Description	Access
GET	/api/cart	Get current user's cart	Auth
POST	/api/cart	Add item to cart	Auth
PUT	/api/cart/:bookId	Update item quantity	Auth
DELETE	/api/cart/:bookId	Remove item from cart	Auth

Favorites (/api/favorites)

Method	Endpoint	Description	Access
GET	/api/favorites	Get user's favorite books	Auth
POST	/api/favorites/:bookId	Toggle favorite on/off	Auth

Orders (/api/orders)

Method	Endpoint	Description	Access
--------	----------	-------------	--------

POST	/api/orders	Checkout — create order from cart	Auth
GET	/api/orders	Get logged-in user's orders	Auth
GET	/api/orders/all	Get all orders (admin)	Admin
PUT	/api/orders/:id/status	Update order status	Admin

5. FRONTEND PAGES & FEATURES

Route	Page	Key Features	Access
/	Home	Hero banner, featured books, category links	Public
/books	All Books	Filters, search, sort, pagination	Public
/books/:id	Book Detail	Full info, add to cart, favorite	Public
/cart	Cart	List, update qty, remove, checkout	Auth
/favorites	Favorites	Wishlist, add to cart, remove	Auth
/orders	My Orders	Order history with status	Auth
/login	Login	Email + password form	Public
/register	Register	Name + email + password form	Public
/admin	Dashboard	Stats: books, orders, users, revenue	Admin
/admin/books	Manage Books	Table with add/edit/delete + image upload	Admin
/admin/categories	Manage Categories	Add / delete categories	Admin
/admin/orders	Manage Orders	View all orders, update status	Admin

6. IMAGE UPLOAD FLOW (Cloudinary)

Book cover images are uploaded through a server-side proxy that securely handles Cloudinary uploads. This avoids exposing Cloudinary API credentials to the client.

Upload Process

Step 1: Admin fills book form and selects an image file.

Step 2: Frontend sends a multipart/form-data POST request with the file and book details.

Step 3: Server middleware (Multer) parses the uploaded file from the request.

Step 4: Cloudinary SDK uploads the file to the cloud and returns a secure URL.

Step 5: The URL is saved in the Book.image field in MongoDB.

Step 6: The frontend displays the image directly from Cloudinary's CDN.

Technical Details

- Multer handles file parsing and temporary storage in memory (memoryStorage)
- Cloudinary SDK (v2) uploads with optional transformations (width, quality)
- supported formats: JPEG, PNG, WebP, GIF
- Maximum file size: 5 MB
- If no image is provided, a default placeholder is used
- On book update, a new image replaces the old one

Cloudinary Configuration (server/.env):

```
CLOUDINARY_CLOUD_NAME=your_cloud_name  
CLOUDINARY_API_KEY=your_api_key  
CLOUDINARY_API_SECRET=your_api_secret
```

7. FOLDER STRUCTURE

```
Book Store/
  server/
    config/
      db.js          # MongoDB connection
    middleware/
      auth.js        # JWT verification middleware
      admin.js       # Admin role check middleware
    models/
      User.js        # User schema
      Book.js        # Book schema
      Category.js    # Category schema
      Cart.js        # Cart schema
      Order.js       # Order schema
    routes/
      auth.js        # Auth routes
      books.js       # Book routes
      categories.js  # Category routes
      cart.js        # Cart routes
      favorites.js   # Favorites routes
      orders.js      # Order routes
    controllers/
      authController.js  # Auth logic
      bookController.js  # Book CRUD logic
      categoryController.js # Category logic
      cartController.js  # Cart logic
      favoriteController.js # Favorites logic
      orderController.js # Order logic
    .env             # Environment variables
    server.js        # Entry point
    package.json
  client/
    public/
    src/
      api/
        index.js     # Axios instance + API calls
      components/
        Navbar.jsx   # Navigation bar
        Footer.jsx   # Footer
        BookCard.jsx # Book card component
        ProtectedRoute.jsx # Auth guard wrapper
        AdminRoute.jsx # Admin guard wrapper
        ...
      context/
        AuthContext.jsx # Auth state
        CartContext.jsx # Cart state
        FavoriteContext.jsx # Favorites state
      pages/
        Home.jsx      # Home page
        AllBooks.jsx  # Browse all books
        BookDetail.jsx # Single book view
        Cart.jsx      # Shopping cart
        Favorites.jsx  # Wishlist
        Login.jsx     # Login form
        Register.jsx  # Registration form
        Orders.jsx    # Order history
      admin/
        Dashboard.jsx # Admin dashboard
        ManageBooks.jsx # Book CRUD
        ManageCategories.jsx # Category management
        ManageOrders.jsx # Order management
      App.jsx         # Root component
      main.jsx        # Entry point
    index.html
    vite.config.js
    tailwind.config.js
    package.json
```

8. ENVIRONMENT VARIABLES

The server requires a .env file at the root of the /server folder with the following variables:

```
# Server Configuration
PORT=5000
# MongoDB Atlas Connection String
MONGODB_URI=mongodb+srv://PrincePtl:princepatel18@prince.hckkerw.mongodb.net/
book_store?retryWrites=true&w=majority&appName=Prince
# JWT Configuration
JWT_SECRET=your_jwt_secret_key_here
JWT_EXPIRE=7d
# Cloudinary Configuration
CLOUDINARY_CLOUD_NAME=your_cloud_name
CLOUDINARY_API_KEY=your_api_key
CLOUDINARY_API_SECRET=your_api_secret
```

9. KEY DELIVERABLES

1. User Authentication

Register, login, JWT token management, protected routes, admin role enforcement.

2. Book Catalog

Browse all books, search by title/author, filter by category and price range, sort by name/price, paginated listing.

3. Shopping Cart

Add/remove/update book quantities, persisted per user in MongoDB, real-time total calculation.

4. Favorites / Wishlist

Toggle favorite status from book cards or detail page, dedicated favorites page.

5. Order System

Checkout flow that creates an order from cart items, deducts stock, tracks order status.

6. Image Upload (Cloudinary)

Server-side image upload via Multer + Cloudinary SDK, secure URL storage in database.

7. Admin Panel

Dashboard with counts/stats, full CRUD for books & categories, order management with status updates.

8. Responsive Design

Mobile-friendly interface using Tailwind CSS breakpoints and utility classes.

READY TO BUILD

Let's turn this plan into reality.